

Plant Disease Detection using CNN and Transfer Learning

AI/ML Specialisation Capstone Project Problem Statement

Field	Details
Project Number	1
Domain	Agriculture / Computer Vision
Recommended Group Size	5 students
Primary Dataset	PlantVillage
Dataset Source URL	https://www.tensorflow.org/datasets/catalog/plant_village
Core Curriculum Alignment	Deep Learning, TensorFlow/Keras, CNN/RNN/Transformers/Generative Models as applicable, Model Optimisation, Deployment, and MLOps.
Expected Prototype	Streamlit web app for demonstration or FastAPI image prediction endpoint with Docker packaging.

1. Background and Context

This capstone project is designed for a group of five learners to demonstrate end-to-end AI/ML capability. It connects model development with data preparation, performance evaluation, deployment readiness, and business interpretation. The project belongs to the Agriculture / Computer Vision domain and uses the PlantVillage dataset.

2. Problem Statement

Farmers and agricultural advisors often need fast preliminary screening of crop leaf diseases. Manual inspection is time-consuming and requires expert availability. The project requires learners to build an image-based classifier that can identify whether a plant leaf is healthy or affected by a disease and classify the disease category where possible.

3. Project Objective

Develop a deployable deep learning application that accepts a leaf image, predicts the most likely plant-disease class, displays confidence scores, and provides a simple interpretation for non-technical users.

4. Target Users and Use Case

Agriculture support teams, agritech field workers, academic learners, and farm advisory teams.

5. Dataset Details

Field	Details
Dataset Name	PlantVillage
Dataset Summary	The PlantVillage dataset contains healthy and unhealthy leaf images divided into plant species and disease categories.
Dataset Access Type	TensorFlow Datasets
Source URL	https://www.tensorflow.org/datasets/catalog/plant_village
Recommended First Step	Download or load a small subset first, validate labels

and file structure, then scale the model training pipeline.

6. Steps to Access the Dataset

- Install the required packages: `pip install tensorflow tensorflow-datasets matplotlib scikit-learn`.
- Import TensorFlow Datasets: `import tensorflow_datasets as tfds`.
- Load the dataset using: `ds, info = tfds.load('plant_village', as_supervised=True, with_info=True)`.
- Create train, validation, and test splits using `tfds` split syntax, for example `train[:80%]`, `train[80%:90%]`, and `train[90%:]`.
- Inspect `info.features['label'].names` to map numeric labels to plant-disease class names.
- Resize images to a fixed input size such as 224 x 224 and normalize pixel values before model training.
- Document the exact classes retained if the group decides to solve a reduced-scope problem, such as tomato-only disease classification.

7. Scope of Work

In Scope

- Use the PlantVillage dataset or a selected crop subset.
- Build a baseline CNN and at least one transfer learning model such as MobileNetV2, ResNet50, or EfficientNet.
- Apply data augmentation, regularization, and validation-based model selection.
- Evaluate class-level performance and misclassification patterns.
- Deploy a simple Streamlit or FastAPI-based image upload application.

Out of Scope

- Real-world disease diagnosis without expert validation.
- Mobile app development beyond a functional prototype.
- Field image collection unless time permits.

8. Recommended Technical Approach

- Define the problem clearly and convert it into a measurable machine learning task.
- Set up a reproducible project repository with folders for data access scripts, notebooks, trained models, reports, and deployment files.
- Perform dataset validation, exploratory analysis, preprocessing, and train-validation-test split creation.
- Build the recommended modelling pipeline: Baseline CNN, MobileNetV2/ResNet50/EfficientNet transfer learning, optional fine-tuning of top layers..
- Track experiments with model configuration, validation metrics, training curves, and error observations.
- Tune key hyperparameters such as learning rate, batch size, number of epochs, optimizer, regularization, and model architecture choices.
- Evaluate on a holdout test set and prepare visual evidence such as confusion matrix, sample predictions, error cases, or mask/box overlays.
- Package the final model into a lightweight demo interface or API and document how to run it.

9. Model Evaluation Plan

Primary evaluation metrics: Accuracy, macro F1-score, per-class precision and recall, confusion matrix, top-3 accuracy, inference latency.

The group should not report only one headline metric. The evaluation must include overall performance, class-wise or segment-wise performance where relevant, failure cases, latency, and a brief explanation of whether the model is usable for the stated user context.

10. Deployment Expectation

Streamlit web app for demonstration or FastAPI image prediction endpoint with Docker packaging.

- The final prototype should load the trained model artifact without requiring retraining.
- The app/API should accept a user input in the expected format and return model output in a clear format.
- The demo should include at least three sample predictions and one known failure case.
- The README should include setup steps, dependencies, and run instructions.

11. Suggested Team Structure for 5 Students

Role	Responsibility
Student 1 - Project Lead & Business Analyst	Owns problem framing, stakeholder context, scope control, final narrative, and presentation coordination.
Student 2 - Data Engineer	Owns dataset access, data validation, preprocessing pipeline, train-validation-test splits, and reproducibility.
Student 3 - Modeling Lead	Owns baseline model, advanced model implementation, training experiments, and tuning strategy.
Student 4 - Evaluation & Research Lead	Owns metrics, error analysis, fairness/ethics checks, benchmarking, and limitation documentation.
Student 5 - Deployment & Documentation Lead	Owns Streamlit/FastAPI demo, Docker or deployment packaging, user guide, and final report formatting.

12. Expected Deliverables

- Problem statement and scope document.
- Data access and preprocessing notebook/script.
- EDA notebook with key observations and data-quality checks.
- Baseline model notebook and advanced model notebook.
- Model evaluation report with metrics, plots, and sample errors.
- Deployment prototype using Streamlit, FastAPI, or equivalent.
- Final presentation deck with problem, method, results, limitations, and future scope.
- GitHub/project folder with README and reproducible instructions.

13. Success Criteria

- Dataset is accessed from a documented public source and loaded reproducibly.
- The group demonstrates a meaningful baseline and an improved advanced model.
- Evaluation metrics are appropriate for the task and interpreted correctly.
- The deployed prototype works on new sample inputs.
- The final explanation clearly connects technical results to user/business value.
- Limitations, risks, and responsible use considerations are stated honestly.

14. Key Risks and Mitigation

Risk	Suggested Mitigation
Dataset images are captured under controlled	Validate assumptions early, document limitations, and

backgrounds and may not generalize to field conditions.	use error analysis to guide improvements.
Some disease classes may be visually similar, causing confusion.	Validate assumptions early, document limitations, and use error analysis to guide improvements.
Large models may have slower inference; use model compression if required.	Validate assumptions early, document limitations, and use error analysis to guide improvements.
Responsible use concern	The output must be framed as educational or advisory support, not a final agricultural diagnosis. The model should display confidence and limitations clearly.

15. Final Presentation Guidance

- Slide 1: Project title, team members, and problem context.
- Slide 2: Dataset source, access method, data size, and sample records/images.
- Slide 3: EDA and preprocessing decisions.
- Slide 4: Baseline model and advanced model architecture.
- Slide 5: Experiment tracking and hyperparameter choices.
- Slide 6: Evaluation results and error analysis.
- Slide 7: Deployment demo and inference workflow.
- Slide 8: Risks, limitations, ethical considerations, and future enhancements.

